Claude Certified Architect — Foundations

CCAR-F Practice Question Set — 60 Items with Answers & Explanations

Six scenario blocks, ten items each, weighted to the exam blueprint.
Distractors are written to be close and realistic, as on the live exam.
Each item is followed by a separate Correct Answer & Explanation section.
Most items select one answer; a few say “Select TWO.”

How to Use This Set

Work each scenario block under time pressure — about two minutes per item, matching the real 60-items-in-120-minutes pace. Answer before reading the explanation. Cover the answer box with a sheet of paper or your hand. For every item, name the trap in each wrong option out loud; that habit is what converts a 689 into a pass. Watch specifically for: prompt-based answers where a deterministic hook/gate is required; over-engineered answers (classifiers, sentiment, extra models) where a description or criteria fix is proportionate; wrong config scope; non-existent CLI features; and "bigger context/model" offered as a fix for attention or calibration problems.

Blueprint coverage of this set: Agentic Architecture & Orchestration (Scenarios 1, 3, 4) · Tool Design & MCP (Scenarios 1, 3, 4) · Claude Code Configuration (Scenarios 2, 4, 5) · Prompt Engineering & Structured Output (Scenarios 5, 6) · Context Management & Reliability (all). Items marked "Select TWO": 24, 57, 60.

Scenario 1 · Customer Support Resolution Agent

Agent SDK agent with MCP tools get_customer, lookup_order, process_refund, escalate_to_human. Target: 80%+ first-contact resolution with sound escalation.

1. Production logs show that in about 9% of refund cases the agent calls process_refund before ever calling get_customer, occasionally refunding the wrong account. The system prompt already instructs the agent to verify identity first. What change most reliably eliminates this?

A. Rewrite the system prompt to state, in bold, that identity verification via get_customer is mandatory before any refund.

B. Add a programmatic prerequisite that blocks process_refund until get_customer has returned a verified customer ID.

C. Add four few-shot examples showing the agent always verifying identity before refunding.

D. Reduce the model temperature to make tool-selection behavior more consistent.

CORRECT ANSWER: B
When a step is safety- or money-critical, only programmatic enforcement (a prerequisite gate/hook) gives a deterministic guarantee. A and C rely on probabilistic LLM compliance, which has a non-zero failure rate that is unacceptable for financial operations. D does not enforce ordering at all and won't reliably fix a compliance requirement.

2. Company policy requires manager approval for any refund over $500, but the agent occasionally issues larger refunds on its own. Which approach best guarantees the policy while still moving the case forward correctly?

A. Add the $500 limit to the system prompt with strong emphasis.

B. Have the agent calculate the amount and decide whether to proceed before calling process_refund.

C. Add a tool-call interception hook that blocks process_refund calls over $500 and redirects to the escalate_to_human workflow.

D. Configure process_refund to silently cap any refund at $500.

CORRECT ANSWER: C
A tool-call interception hook both enforces the rule deterministically and routes the blocked action to the correct alternative workflow (human escalation), which is the complete desired behavior. A and B are probabilistic. D enforces a limit but produces a wrong business outcome (silently altering the refund) rather than escalating for approval.

3. A customer's very first message reads: "Don't give me a bot. Connect me to a real person now." What should the agent do?

A. Attempt to resolve the underlying issue first and escalate only if it cannot.

B. Ask a few questions to understand the problem before deciding whether to escalate.

C. Honor the explicit request and escalate to a human immediately.

D. Explain that the assistant can usually resolve issues faster, then proceed.

CORRECT ANSWER: C
An explicit customer request for a human is a first-class escalation trigger and should be honored immediately, without first attempting investigation. A, B, and D all delay or override a clear request.

4. A customer writes: "This is absurd, it's the third time my order arrived damaged!" The issue is a standard damaged-item reshipment the agent is fully capable of handling, and the customer has not asked for a human. What is the best response?

A. Escalate to a human immediately because the sentiment is strongly negative.

B. Acknowledge the frustration and offer to process the reshipment now, escalating only if the customer still wants a human.

C. Run sentiment analysis and escalate if the anger score exceeds a threshold.

D. Ask the customer to confirm they are willing to keep working with the assistant before continuing.

CORRECT ANSWER: B
Frustration is not the same as an explicit request for a human, and the issue is within the agent's capability, so the agent should acknowledge and offer resolution, escalating only if the customer reiterates. A and C treat sentiment as an escalation trigger, but sentiment does not correlate with case complexity. D adds needless friction.

5. The agent frequently confuses two tools when customers reference orders. Both descriptions are minimal ("Retrieves customer information" and "Retrieves order details") and both accept similar identifier formats. What is the most effective first step to improve tool selection?

A. Add five to eight few-shot examples showing order queries routed to the order tool.

B. Expand each tool's description to include input formats, example queries, edge cases, and when to use it versus the similar tool.

C. Add a pre-turn routing layer that inspects the message and pre-selects the tool by keyword.

D. Merge the two tools into one lookup_entity tool that decides the backend internally.

CORRECT ANSWER: B
Tool descriptions are the primary mechanism the model uses for selection; minimal descriptions are the root cause, and enriching them is the low-effort, high-leverage first step. Few-shot (A) adds token overhead without fixing the root cause. A routing layer (C) is over-engineered and bypasses the model's language understanding. Consolidation (D) is a larger architectural change than a "first step" warrants.

6. When the agent calls get_customer with only a name, the tool returns three different matching accounts. What should the agent do?

A. Select the account with the most recent activity.

B. Ask the customer for an additional identifier, such as an email or order number, to disambiguate.

C. Escalate to a human because multiple matches indicate a complex case.

D. Proceed with all three accounts and let downstream tools filter them.

CORRECT ANSWER: B
Multiple matches call for clarification via an additional identifier rather than a heuristic guess (A, D), which risks acting on the wrong account. Multiple matches alone are not a complexity signal that warrants escalation (C); requesting one more identifier is the proportionate resolution.

7. A single customer message contains three distinct issues: a refund request, a wrong shipping address, and a coupon that failed to apply. What is the best way to handle it?

A. Resolve the first issue and ask the customer to open separate tickets for the others.

B. Decompose the message into three items, investigate each (in parallel where possible) using shared context, then synthesize one unified resolution.

C. Escalate the whole conversation because multi-issue requests exceed the agent's scope.

D. Address only the issue whose wording sounds most urgent.

CORRECT ANSWER: B
Multi-concern requests should be decomposed into distinct items, each investigated on shared context, then synthesized into a single resolution. A shifts work to the customer, C escalates unnecessarily, and D ignores parts of the request.

8. The agent must escalate mid-conversation to a human agent who has no access to the chat transcript. What should it pass to the human?

A. The complete raw conversation transcript.

B. A brief apology and the customer's name.

C. A structured handoff summary with customer ID, root cause, refund amount, and recommended action.

D. Only the customer's most recent message.

CORRECT ANSWER: C
A structured handoff (customer ID, root cause, amount, recommended action) gives the human exactly what they need to act without the transcript. The raw transcript (A) is unstructured and slow to parse; B and D omit essential context.

9. Over a long conversation, progressive summarization has reduced the case to “customer wants a refund,” dropping that it was $129.99 for a specific damaged order with a same-day reship promised by an earlier agent. What is the best fix?

A. Increase max_tokens so more of the raw history stays in context.

B. Extract transactional facts (amounts, order IDs, dates, promises) into a persistent case-facts block included in every prompt, kept outside the summarized history.

C. Disable summarization and always send the entire history verbatim.

D. Ask the customer to restate all the details.

CORRECT ANSWER: B
Summarization tends to flatten precise numbers, dates, and commitments; extracting them into a persistent case-facts block outside the summary preserves them regardless of how history is condensed. A and C are brittle and token-inefficient, and D degrades the customer experience.

10. In the agent's control loop, which condition correctly indicates that the agent has finished and its final message should be returned to the customer?

A. The assistant message ends with a phrase like “Is there anything else I can help with?”

B. The response's stop_reason is “end_turn.”

C. The loop has run a fixed maximum number of iterations.

D. The most recent tool call returned without an error.

CORRECT ANSWER: B
Loop termination is determined solely by stop_reason: continue on “tool_use,” stop on “end_turn.” Parsing assistant text (A), using an iteration cap as the primary stop (C), or checking tool error status (D) are all named anti-patterns.

Scenario 2 · Code Generation with Claude Code

Team uses Claude Code for generation, refactoring, debugging, and docs, integrated via custom slash commands and CLAUDE.md, choosing plan mode versus direct execution.

11. You need to add a single null check to one function that currently throws on empty input, and you already have the stack trace pinpointing it. Which mode is appropriate?

A. Plan mode, to design the change carefully.

B. Direct execution.

C. Fork the session to compare alternative fixes.

D. Write a full test suite before touching the code.

CORRECT ANSWER: B
Direct execution fits simple, well-scoped changes with clear intent, such as a single-file fix with a known stack trace. Plan mode (A) and session forking (C) are for architectural or multi-approach work; a full pre-emptive test suite (D) is disproportionate here.

12. You must migrate the codebase from one HTTP client library to another, affecting roughly 45 files, and there are two viable migration strategies with different infrastructure implications. What is the best approach?

A. Use direct execution and fix files one by one as compile errors surface.

B. Enter plan mode to explore dependencies and choose an approach before making changes.

C. Use direct execution with a single comprehensive instruction specifying every file's changes upfront.

D. Begin in direct execution and switch to plan mode only if unexpected complexity appears.

CORRECT ANSWER: B
Plan mode is designed for large-scale, multi-file changes with multiple valid approaches and architectural decisions. A risks late-discovered rework, C assumes the right structure is already known, and D wrongly defers planning when the complexity is already stated in the requirements.

13. You want a custom /migrate-check slash command that runs the team's migration checklist and is available to every developer automatically when they clone or pull the repository. Where should the command live?

A. In each developer's ~/.claude/commands/ directory.

B. In the project's .claude/commands/ directory.

C. In a “commands” array inside .claude/config.json.

D. As a section in the root CLAUDE.md file.

CORRECT ANSWER: B
Project-scoped commands in .claude/commands/ are version-controlled and reach everyone on clone/pull. A is personal and not shared. C references a file/mechanism that does not exist. D is for project context, not command definitions.

14. A new teammate reports that Claude ignores the “always run the formatter before committing” instruction that works reliably for you. What is the most likely cause?

A. The instruction is in your ~/.claude/CLAUDE.md (user-level) and therefore isn't shared through version control.

B. The teammate needs to run /compact to load the instruction.

C. CLAUDE.md supports only one active instruction at a time.

D. Formatting rules must be a slash command, not CLAUDE.md text.

CORRECT ANSWER: A
User-level configuration is not shared via version control, so a teammate won't receive it; moving the instruction to project-level CLAUDE.md fixes this. B, C, and D describe behaviors that don't exist. Use /memory to verify which memory files are loaded.

15. Despite a detailed prose description, Claude keeps producing a date-normalization helper with inconsistent output formatting between runs. What is the most effective way to communicate the expected transformation?

A. Expand the prose description with more precise wording.

B. Provide two or three concrete input/output examples of the transformation.

C. Lower the temperature to reduce variability.

D. Ask Claude to restate the requirements before coding.

CORRECT ANSWER: B
Concrete input/output examples are the most effective way to convey a transformation when prose is interpreted inconsistently. More prose (A) tends to reproduce the ambiguity, and C and D don't pin down the expected format.

16. You're about to implement a caching layer in a domain you don't know well and you're worried about missing considerations like invalidation and cache stampede. What technique best surfaces these before you commit to an implementation?

A. Have Claude implement a first version and fix issues as they appear.

B. Use the interview pattern: have Claude ask you questions to surface design considerations before implementing.

C. Implement it, then run a review pass to catch problems.

D. Write one exhaustive specification covering everything you can currently think of.

CORRECT ANSWER: B
The interview pattern has Claude proactively raise considerations the developer may not have anticipated, which is exactly the gap in an unfamiliar domain. A and C surface issues only after implementation, and D is limited to what you already know to specify.

17. A review surfaces three issues: two are independent formatting nits, and the third is a bug whose fix changes a function signature that a second issue depends on. How should you direct the fixes?

A. Provide all three issues in one message regardless of their relationships.

B. Fix every issue in its own separate message regardless of their relationships.

C. Address the interacting bug and its dependent issue together in one detailed message, and handle the independent nits separately.

D. Fix only the bug and leave the nits unaddressed.

CORRECT ANSWER: C
Interacting problems should be fixed together so the model can reason about them jointly; independent problems can be handled separately. A and B apply one rigid rule to both kinds, and D ignores real issues.

18. Configuration files matching *.config.ts are scattered across many packages, and you want Claude to automatically apply one convention to all of them regardless of directory. What is the most maintainable approach?

A. Create a .claude/rules/ file with YAML frontmatter whose paths glob is **/*.config.ts.

B. Place a CLAUDE.md in each package directory describing the convention.

C. Create a skill developers invoke manually when editing config files.

D. Add one large section to the root CLAUDE.md and rely on Claude to infer when it applies.

CORRECT ANSWER: A
Glob-scoped rules in .claude/rules/ apply automatically by file type across any directory, which is ideal for scattered files. Subdirectory CLAUDE.md (B) is directory-bound, a manual skill (C) isn't automatic, and inference from a monolithic file (D) is unreliable.

19. A “codebase-map” skill produces long, verbose output that clutters the main conversation and consumes context. What is the best configuration change?

A. Add context: fork to the skill's SKILL.md so it runs in an isolated sub-agent context.

B. Add allowed-tools to the skill to restrict which tools it can call.

C. Move the skill from .claude/skills/ to ~/.claude/skills/.

D. Run /compact after every invocation of the skill.

CORRECT ANSWER: A
context: fork runs the skill in an isolated sub-agent so its verbose output doesn't pollute the main session. allowed-tools (B) restricts tools, not output volume; changing scope (C) doesn't isolate output; and /compact (D) is a reactive cleanup, not a structural fix.

20. You investigated a large codebase in a named session, then made substantial code changes, so the file contents captured in that session's tool results are now stale. What is the most reliable way to continue the work?

A. Resume the prior session with --resume and keep going.

B. Fork the session to create a branch from the prior analysis.

C. Start a new session and inject a structured summary of prior findings, noting which files changed.

D. Increase the context window so the stale results don't get dropped.

CORRECT ANSWER: C
When prior tool results are stale, starting fresh with an injected structured summary is more reliable than resuming, which would carry the stale results forward. Forking (B) also inherits the stale baseline, and a larger context (D) preserves rather than corrects the stale data.

Scenario 3 · Multi-Agent Research System

A coordinator delegates to search, document-analysis, synthesis, and report subagents to produce comprehensive, cited reports.

21. Your coordinator never delegates; it attempts every step itself instead of spawning subagents. What is the most likely configuration problem?

A. The coordinator's allowedTools does not include “Task.”

B. The subagents are missing system prompts.

C. tool_choice is set to “any.”

D. The coordinator's max_tokens is too low to delegate.

CORRECT ANSWER: A
Subagents are spawned via the Task tool, and the coordinator can only invoke them if allowedTools includes “Task.” Missing subagent prompts (B) would affect subagent quality, not spawning; tool_choice (C) and max_tokens (D) don't govern the ability to delegate.

22. The synthesis subagent's reports omit key details that the search subagent clearly found, even though the coordinator invoked synthesis after search completed. What is the most likely cause?

A. The synthesis subagent has access to too many tools.

B. The search findings were not included in the synthesis subagent's prompt; subagents do not automatically inherit the coordinator's or other subagents' context.

C. The synthesis subagent needs a larger max_tokens.

D. The coordinator ran the subagents sequentially instead of in parallel.

CORRECT ANSWER: B
Subagents operate with isolated context and don't inherit prior findings automatically; the coordinator must pass search results directly in the synthesis subagent's prompt. Tool count (A), max_tokens (C), and execution ordering (D) don't explain missing upstream findings.

23. Researching “the state of renewable energy,” every subagent succeeds, yet the final reports cover only solar. The coordinator's logs show it decomposed the topic into “solar panel efficiency,” “solar farm economics,” and “residential solar,” omitting wind, hydro, and geothermal. What is the root cause?

A. The search subagent's queries are not comprehensive enough.

B. The coordinator's task decomposition is too narrow, so subagent assignments miss whole domains of the topic.

C. The synthesis subagent is dropping non-solar findings.

D. The document-analysis subagent's relevance criteria are too restrictive.

CORRECT ANSWER: B
The logs implicate the coordinator: it never assigned wind, hydro, or geothermal, so the subagents executed correct work on an incomplete plan. A, C, and D wrongly blame downstream agents that performed correctly within their assigned scope.

24. The document-analysis subagent's backend service fails partway through a task. Which TWO practices best enable the coordinator to recover intelligently? (Select TWO.)

A. Return structured error context including the failure type, what was attempted, and any partial results.

B. Distinguish access failures (the query could not run) from valid empty results (the query ran but found nothing).

C. Return a generic “analysis unavailable” status once retries are exhausted.

D. Catch the failure and return an empty result set marked as successful.

E. Propagate the exception to a top-level handler that terminates the entire research workflow.

CORRECT ANSWER: A AND B
Structured error context lets the coordinator decide whether to retry, try an alternative, or proceed with partial results, and distinguishing access failures from valid empty results prevents wrong recovery decisions. C hides context, D silently suppresses the error, and E needlessly kills the whole workflow — all named anti-patterns.

25. The synthesis subagent frequently needs simple fact checks (dates, names, statistics) while combining findings. Currently it returns control to the coordinator, which invokes the search subagent and re-invokes synthesis, adding round-trips and ~40% latency. About 85% of these checks are simple; 15% need deeper investigation. What is the best approach?

A. Give the synthesis subagent a scoped verify_fact tool for simple lookups, while complex verifications still route through the coordinator.

B. Give the synthesis subagent access to all search tools so it can verify anything directly.

C. Have synthesis accumulate all verification needs and send them to the coordinator as one batch at the end.

D. Have the search subagent cache extra context around each source up front to anticipate verification needs.

CORRECT ANSWER: A
A scoped verify_fact tool serves the 85% common case with least privilege while preserving coordinator routing for the complex 15%. B over-provisions and breaks separation of concerns, C creates blocking dependencies when later steps rely on earlier verified facts, and D relies on speculative caching that can't predict needs.

26. The coordinator needs to research four independent subtopics and wants to minimize total latency. How should it spawn the subagents?

A. Invoke the subagents one at a time across separate turns.

B. Emit multiple Task tool calls in a single coordinator response so the subagents run in parallel.

C. Combine all four subtopics into a single subagent's prompt.

D. Use fork_session to create four branches.

CORRECT ANSWER: B
Emitting multiple Task calls in one response spawns subagents in parallel, minimizing latency for independent work. Sequential turns (A) are slower, one combined prompt (C) loses parallelism and partitioning, and fork_session (D) is for exploring divergent approaches from a shared baseline, not parallel subtopic research.

27. In the final reports, claims appear without their sources; attribution is being lost somewhere between research and synthesis. What is the best fix?

A. Add a citation-formatting pass at the very end of report generation.

B. Require subagents to output structured claim-source mappings (claim, evidence excerpt, source name/URL, date) that downstream agents preserve through synthesis.

C. Instruct the synthesis subagent to remember the sources it used.

D. Increase the context window so nothing is dropped during synthesis.

CORRECT ANSWER: B
Attribution is lost when findings are compressed without preserving claim-source mappings; requiring structured mappings that are preserved through synthesis fixes the root cause. A late formatting pass (A) can't recover lost links, an instruction to “remember” (C) is unreliable, and more context (D) doesn't preserve mappings.

28. Two credible sources report materially different figures for the same market size. How should the synthesis subagent handle this?

A. Choose the larger figure.

B. Keep only the figure from the more recent source.

C. Present both figures with source attribution and annotate the conflict, structuring the report to distinguish contested from well-established findings.

D. Report the average of the two figures.

CORRECT ANSWER: C
Conflicting values from credible sources should be preserved and annotated with attribution rather than arbitrarily resolved. Picking one (A, B) or averaging (D) discards information and can misrepresent the evidence.

29. The system flags two sources as contradictory on an unemployment statistic, but one dataset is from 2023 and the other from 2025 — they aren't actually in conflict. What design change best prevents this misinterpretation?

A. Require subagents to include publication or data-collection dates in their structured outputs.

B. Configure the system to keep only the newest source for any statistic.

C. Add a conflict_detected boolean to every finding.

D. Instruct the synthesis subagent to ignore statistics entirely.

CORRECT ANSWER: A
Requiring dates lets the system interpret temporal differences correctly instead of reading them as contradictions. Keeping only the newest source (B) discards valid historical data, a boolean alone (C) doesn't supply the temporal context, and ignoring statistics (D) defeats the purpose.

30. After synthesis, the coordinator determines that one subtopic is only thinly covered. What is the best pattern to improve the report?

A. Ship the report with a disclaimer about the thin subtopic.

B. Re-delegate targeted queries to the search and analysis subagents, then re-invoke synthesis, repeating until coverage is sufficient.

C. Ask the end user to research the thin subtopic themselves.

D. Permanently increase the number of subagents in the pipeline.

CORRECT ANSWER: B
An iterative refinement loop — evaluate synthesis for gaps, re-delegate targeted queries, re-synthesize — is the intended coverage mechanism. A and C abandon quality, and D adds fixed overhead without targeting the actual gap.

Scenario 4 · Developer Productivity with Claude

An Agent SDK tool helps engineers explore unfamiliar codebases and automate tasks using built-in tools (Read, Write, Bash, Grep, Glob) and MCP servers.

31. You need to find every caller of a function named computeTax across the codebase. Which built-in tool is the right choice?

A. Glob with a pattern like **/*.ts.

B. Grep for the string computeTax.

C. Read every file and scan for the function.

D. Bash with a recursive directory listing.

CORRECT ANSWER: B
Grep searches file contents for patterns such as function names, which is exactly finding callers. Glob (A) matches file paths, not contents; reading everything (C) is inefficient; and a directory listing (D) doesn't search contents.

32. You need to locate all test files matching the pattern *.test.tsx anywhere in the repository. Which built-in tool fits best?

A. Grep for ".test.tsx."

B. Glob with **/*.test.tsx.

C. Read the top-level test directory.

D. Edit to open matching files.

CORRECT ANSWER: B
Glob matches files by name/extension patterns, which is the correct tool for finding files by a naming pattern. Grep (A) searches contents, reading one directory (C) misses scattered files, and Edit (D) modifies files rather than finding them.

33. An Edit operation fails because its anchor text, "return result;", appears many times in the file, so the match is not unique. What is the most reliable way to make the change?

A. Retry the same Edit with the identical anchor text.

B. Use Read to load the full file, then Write the modified contents.

C. Use Grep to count the occurrences first.

D. Use Bash stream editing to force the replacement.

CORRECT ANSWER: B
When Edit can't find a unique anchor, the reliable fallback is Read the full file, then Write the modified version. Retrying the same Edit (A) fails again, counting occurrences (C) doesn't perform the edit, and forcing a stream edit (D) is error-prone and not the recommended approach.

34. What is the best strategy for building understanding of a large, unfamiliar codebase?

A. Read all files upfront to load complete context before answering questions.

B. Start with Grep to find entry points, then use Read to follow imports and trace flows incrementally.

C. Ask the agent to summarize the codebase from typical patterns for that kind of project.

D. Glob every file and Read them in alphabetical order.

CORRECT ANSWER: B
Incremental exploration — Grep for entry points, then Read to follow imports — builds accurate understanding without exhausting context. Reading everything (A, D) wastes context and dilutes attention, and relying on "typical patterns" (C) invents details rather than reading the actual code.

35. Each session, the agent wastes several turns exploring the database to rediscover its schema. What MCP design best reduces this?

A. Add more search tools so exploration is faster.

B. Expose the schema as an MCP resource (a content catalog) so the agent has visibility without exploratory tool calls.

C. Paste the full schema into the system prompt on every request.

D. Increase the context window to hold more exploration output.

CORRECT ANSWER: B
MCP resources expose content catalogs (like schemas) so agents gain visibility without exploratory calls. More search tools (A) don't remove the need to explore, hardcoding the schema in every prompt (C) is brittle and token-heavy, and a bigger context (D) doesn't reduce exploration.

36. The agent keeps using the built-in Grep tool instead of your more capable MCP code-search tool. What is the best fix?

A. Remove the agent's access to Grep entirely.

B. Enhance the MCP tool's description to clearly explain its capabilities and outputs so the agent prefers it.

C. Rename the built-in Grep tool.

D. Force tool_choice to the MCP tool on every turn.

CORRECT ANSWER: B
Enhancing the MCP tool's description so its capabilities are clear leads the agent to prefer it over a built-in, addressing the root cause. Removing Grep (A) is heavy-handed and removes a useful tool, renaming Grep (C) doesn't clarify the MCP tool's value, and forcing tool_choice every turn (D) is inflexible and wrong for turns where Grep is actually better.

37. You need a standard Jira integration for the team. Should you build a custom MCP server or use an existing one?

A. Build a custom MCP server to retain full control over the integration.

B. Use an existing community MCP server for the standard Jira integration, reserving custom servers for team-specific workflows.

C. Skip MCP and call the Jira REST API directly from the agent.

D. Expose Jira only as an MCP resource, with no tools.

CORRECT ANSWER: B
For standard integrations, existing community MCP servers are preferred; custom servers are reserved for team-specific workflows. Building custom for a standard need (A) is wasted effort, bypassing MCP (C) loses the integration model's benefits, and resource-only (D) can't perform the actions Jira work requires.

38. An agent has access to 18 tools and frequently selects the wrong one. What is the best improvement to selection reliability?

A. Add few-shot examples covering all 18 tools.

B. Restrict the agent to the four or five tools relevant to its role.

C. Increase the agent's max_tokens.

D. Set tool_choice to “any.”

CORRECT ANSWER: B
Too many tools increases decision complexity and degrades selection; scoping the agent to only the tools its role needs is the fix. Few-shot for all 18 (A) adds overhead without reducing complexity, max_tokens (C) is unrelated, and tool_choice “any” (D) forces a tool call but doesn't help pick the right one.

39. A generic fetch_url tool is being misused to pull arbitrary or invalid URLs. What is the best fix?

A. Replace it with a constrained load_document tool that validates document URLs.

B. Add a warning in the system prompt about which URLs are allowed.

C. Remove all URL-related tools from the agent.

D. Wrap fetch_url in a hook that only logs its usage.

CORRECT ANSWER: A
Replacing a generic tool with a constrained, purpose-specific alternative (load_document with URL validation) prevents misuse by design. A prompt warning (B) is probabilistic, removing all URL tools (C) is over-broad, and a logging-only hook (D) records misuse without preventing it.

40. During a long codebase-exploration session, Claude begins giving inconsistent answers and referencing “typical patterns” instead of the specific classes it identified earlier. What is the best mitigation?

A. Restart the session from scratch for every new question.

B. Have the agent maintain a scratchpad file of key findings and reference it for subsequent questions.

C. Increase the temperature to encourage more varied answers.

D. Re-ask each question multiple times and keep the most common answer.

CORRECT ANSWER: B
A scratchpad file persists key findings across context boundaries and counteracts context degradation in long sessions. Restarting constantly (A) discards useful state, higher temperature (C) worsens consistency, and re-asking (D) doesn't restore lost specifics.

Scenario 5 · Claude Code for Continuous Integration

Claude Code runs automated reviews, generates tests, and comments on pull requests, with prompts designed for actionable feedback and minimal false positives.

41. A CI job runs claude "Review this PR for bugs" and hangs indefinitely; logs show it's waiting for interactive input. What is the correct fix?

A. Run claude -p "Review this PR for bugs."

B. Set the environment variable CLAUDE_HEADLESS=true.

C. Run claude --batch "Review this PR for bugs."

D. Redirect stdin from /dev/null.

CORRECT ANSWER: A
The -p (or --print) flag runs Claude Code non-interactively: it processes the prompt, prints the result, and exits, which is what CI needs. CLAUDE_HEADLESS (B) and --batch (C) don't exist, and the /dev/null redirect (D) is a workaround rather than the documented approach.

42. You need the CI review's output as machine-parseable findings so a script can post them as inline PR comments. Which option achieves this?

A. Use --output-format json together with --json-schema.

B. Use --print alone and parse the prose output.

C. Add --verbose to include more detail.

D. Use --format markdown.

CORRECT ANSWER: A
--output-format json with --json-schema enforces structured, schema-compliant output suitable for automated posting. Parsing prose from --print (B) is unreliable, --verbose (C) adds noise not structure, and --format markdown (D) isn't the mechanism for enforced structured output.

43. The same Claude session that generated a piece of code is then asked to review it and misses obvious bugs. What is the most effective fix?

A. Instruct the session to think more carefully and enable extended thinking during review.

B. Use a second, independent Claude instance without the generator's reasoning context to perform the review.

C. Increase max_tokens so the review can be more thorough.

D. Have the session review the code three times and take the majority verdict.

CORRECT ANSWER: B
A model retains its generation reasoning and is less likely to question its own decisions, so an independent review instance is more effective than self-review or extended thinking. A and D keep the same reasoning context, and max_tokens (C) doesn't address the self-review limitation.

44. A single-pass review of a 20-file PR produces inconsistent results: detailed feedback on some files, superficial comments on others, missed bugs, and contradictory findings (flagging a pattern in one file while approving identical code in another). How should you restructure the review?

A. Split the review into per-file passes for local issues plus a separate integration pass for cross-file data flow.

B. Move to a model with a larger context window so all 20 files get attention at once.

C. Require developers to break the PR into smaller submissions before review.

D. Run three full-PR passes and only flag issues that appear in at least two runs.

CORRECT ANSWER: A
Per-file passes give consistent local depth while a separate integration pass catches cross-file issues, addressing attention dilution directly. A larger context (B) doesn't fix attention quality, splitting PRs (C) shifts burden to developers, and consensus voting (D) suppresses real bugs caught only intermittently.

45. Your comment-accuracy check has a high false-positive rate, flagging perfectly good comments. What most effectively improves precision?

A. Add the instruction "only report high-confidence issues."

B. Add the instruction "be conservative when flagging."

C. Replace the vague check with an explicit criterion: flag a comment only when its claimed behavior contradicts the actual code behavior.

D. Lower the model temperature.

CORRECT ANSWER: C
Explicit, categorical criteria improve precision far more than confidence- or conservatism-based instructions, which don't give the model a concrete rule. A and B are the vague instructions the guide warns against, and temperature (D) doesn't define what counts as an issue.

46. One review category, style suggestions, has roughly a 60% false-positive rate and is eroding developer trust in the tool overall. What is the best immediate action while you work on improving it?

A. Keep the category enabled; developers can ignore the noise.

B. Temporarily disable the high-false-positive category to restore trust while you refine its prompt.

C. Remove the entire automated review tool until everything is improved.

D. Lower the model temperature to reduce false positives.

CORRECT ANSWER: B
Temporarily disabling a noisy category restores trust in the accurate categories while you improve the problematic one. Leaving it on (A) continues eroding trust, removing the whole tool (C) is disproportionate, and temperature (D) doesn't fix a criteria problem.

47. Two workflows currently use real-time Claude calls: a blocking pre-merge check developers wait on, and an overnight technical-debt report reviewed the next morning. Your manager wants both moved to the Message Batches API for its 50% savings. How should you evaluate this?

A. Move only the overnight report to batch; keep the pre-merge check synchronous.

B. Move both workflows to batch and poll for completion.

C. Keep both workflows synchronous to avoid batch result-ordering problems.

D. Move both to batch with a timeout fallback to synchronous if batches run long.

CORRECT ANSWER: A
Batch has no latency SLA and can take up to 24 hours, making it unsuitable for a blocking pre-merge check but ideal for an overnight job. B risks unacceptable waits on the blocking check, C is based on a false ordering concern (custom_id handles correlation), and D adds needless complexity.

48. Re-running the CI review after each new commit causes duplicate PR comments for issues already reported. What is the best fix?

A. Include the prior review findings in context and instruct Claude to report only new or still-unaddressed issues.

B. Disable the review on all commits after the first.

C. Write all comments to a separate log file instead of the PR.

D. Increase the context window so the model remembers earlier comments.

CORRECT ANSWER: A
Providing prior findings and instructing the model to report only new or unresolved issues prevents duplicates while preserving review value. Disabling later reviews (B) misses new problems, moving comments off the PR (C) defeats the purpose, and a bigger context (D) doesn't tell the model to deduplicate.

49. Your CI-generated tests are low-value and ignore the project's established fixtures and testing standards. What most improves generation quality?

A. Increase max_tokens so more tests are generated.

B. Document testing standards, valuable-test criteria, and available fixtures in CLAUDE.md so CI-invoked Claude uses them.

C. Switch test generation to the Message Batches API.

D. Ask for more tests per run.

CORRECT ANSWER: B
CLAUDE.md is the mechanism for supplying project context (standards, fixtures, criteria) to CI-invoked Claude, which raises test quality and reduces low-value output. More tokens or more tests (A, D) increase volume, not quality, and batching (C) affects cost/latency, not test relevance.

50. In a nightly batch of 500 test-generation jobs, 12 fail because those source files exceed the context limit. What is the best recovery?

A. Resubmit the entire batch of 500.

B. Resubmit only the 12 failed items, identified by custom_id, after chunking the oversized files.

C. Switch all 500 jobs to synchronous calls.

D. Accept the 12 failures and move on.

CORRECT ANSWER: B
Batch failures are handled by resubmitting only the failed items (by custom_id) with appropriate modifications, such as chunking documents that exceeded the limit. Resubmitting all 500 (A) wastes cost, switching to synchronous (C) is unnecessary, and ignoring failures (D) leaves work undone.

Scenario 6 · Structured Data Extraction

A system extracts data from unstructured documents, validates output against JSON schemas, maintains high accuracy, and integrates with downstream systems.

51. You need guaranteed schema-valid JSON output with no syntax errors. What is the most reliable approach?

A. Ask for JSON in the prompt and parse the model's text response.

B. Define an extraction tool with a JSON input schema and read the structured data from the tool_use response.

C. Post-process the model's text with regex to repair malformed JSON.

D. Lower the temperature to reduce formatting mistakes.

CORRECT ANSWER: B
tool_use with a JSON schema is the most reliable way to guarantee schema-compliant output and eliminates JSON syntax errors. Prompt-and-parse (A) and regex repair (C) still allow malformed output, and lower temperature (D) reduces but doesn't eliminate syntax errors.

52. The model sometimes invents a tax_id value for documents that don't contain one. What schema change best prevents this fabrication?

A. Make tax_id a required field so the model always populates it.

B. Make tax_id optional/nullable so the model returns null when the value is absent.

C. Add a note in the prompt that tax_id is very important.

D. Add a few-shot example that includes a representative tax_id.

CORRECT ANSWER: B
Making a field optional/nullable when the source may lack it prevents the model from fabricating a value to satisfy a required field. Requiring it (A) causes the exact fabrication you're trying to stop, and A prompt emphasis (C) or an example value (D) can encourage invention rather than a null.

53. A document_type field is usually one of five known values but occasionally something new or genuinely ambiguous. What is the best schema design?

A. Use a free-text string field.

B. Use an enum limited to the five known values.

C. Use an enum of the five values plus “other” with a detail string, and “unclear” for ambiguous cases.

D. Use a separate boolean field for each of the five types.

CORRECT ANSWER: C
An enum with “other” plus a detail string handles extensibility, and “unclear” handles ambiguity, matching the recommended pattern. Free text (A) loses structure, a closed enum (B) can't represent new or ambiguous cases, and per-type booleans (D) are awkward and allow contradictory combinations.

54. Your strict tool_use schema eliminated JSON syntax errors, yet extracted line items still sometimes don't sum to the stated total. Why does this happen, and what is the best fix?

A. The schema is malformed; rewrite it.

B. Strict schemas eliminate syntax errors but not semantic ones; extract calculated_total alongside stated_total and flag any mismatch.

C. Set tool_choice to “any” to force better output.

D. Increase max_tokens so the model can compute more carefully.

CORRECT ANSWER: B
Strict schemas guarantee well-formed structure, not correct values; catching semantic errors requires validation such as comparing a calculated total to the stated total. A misdiagnoses the cause, and tool_choice (C) and max_tokens (D) don't validate values.

55. An extraction fails schema validation because a date is in the wrong format. What is the best recovery?

A. Discard the extraction and move on to the next document.

B. Send a follow-up request with the original document, the failed extraction, and the specific validation error so the model can self-correct.

C. Retry the identical request unchanged.

D. Route the document to a human immediately.

CORRECT ANSWER: B
Retry-with-error-feedback — resending the document, the failed extraction, and the specific error — guides the model to fix format/structural errors like a bad date. Discarding (A) loses data, an identical retry (C) is likely to repeat the error, and human routing (D) is premature for a correctable format issue.

56. An extraction returns null for contract_end_date because that date isn't in the provided document; it lives in a separate appendix that wasn't supplied. Will a retry with error feedback help?

A. Yes, retry with stronger emphasis on finding the date.

B. No; the information is absent from the source, so retries won't help. Provide the appendix or route to human review.

C. Yes, add a few-shot example of a contract with an end date.

D. Yes, retry with a larger model.

CORRECT ANSWER: B
Retries help with format or structural errors but not when the required information simply isn't present in the source. Emphasis (A), examples (C), or a larger model (D) can't extract a value that isn't in the document.

57. Extraction works well on structured-table invoices but frequently returns nulls on narrative, prose-style invoices. Which TWO changes best improve handling across formats? (Select TWO.)

A. Add few-shot examples demonstrating correct extraction from varied formats, including narrative prose and different citation/layout styles.

B. Include format-normalization rules in the prompt alongside the strict output schema.

C. Make all schema fields required so the model must fill them.

D. Increase max_tokens for prose documents.

E. Require all incoming invoices to be reformatted as tables first.

CORRECT ANSWER: A AND B
Few-shot examples spanning formats help the model generalize to varied structures, and format-normalization rules in the prompt handle inconsistent source formatting alongside the schema. Making fields required (C) causes fabrication, more tokens (D) doesn't address format understanding, and reformatting all inputs (E) is impractical and shifts the burden upstream.

58. Your extractor reports 97% overall accuracy and you want to reduce human review. What should you verify first?

A. Nothing; 97% overall accuracy is high enough to automate.

B. Analyze accuracy by document type and by field to confirm no segment performs poorly before reducing review.

C. Raise the review sample to 100% permanently.

D. Move extraction to batch processing to cut costs first.

CORRECT ANSWER: B
Aggregate accuracy can mask poor performance on specific document types or fields, so you must validate by segment before automating. Trusting the aggregate (A) is exactly the trap, 100% review (C) defeats automation, and batching (D) is unrelated to accuracy validation.

59. You want ongoing error monitoring of high-confidence extractions and a way to route uncertain extractions to humans. What is the best design?

A. Randomly sample all extractions uniformly for review.

B. Use stratified random sampling of high-confidence extractions to measure error rates, and calibrate field-level confidence on a labeled validation set to route low-confidence or ambiguous items to human review.

C. Review only the extractions the model self-reports as low confidence, trusting the uncalibrated scores.

D. Review a fixed set of the first ten extractions each day.

CORRECT ANSWER: B
Stratified sampling of high-confidence extractions detects error rates and novel patterns, while confidence calibrated on labeled data provides a reliable routing signal. Uniform sampling (A) is less efficient for finding rare high-confidence errors, uncalibrated self-reports (C) are unreliable, and a fixed daily set (D) isn't representative.

60. Your team plans to use the Message Batches API for a weekly compliance audit of 10,000 documents. Which TWO statements are accurate reasons this is a good fit or true properties of the API? (Select TWO.)

A. It offers about 50% cost savings compared with synchronous calls.

B. It guarantees results within a fixed, low-latency SLA.

C. custom_id correlates each response with its request, enabling targeted resubmission of failures.

D. It supports multi-turn tool calling within a single batch request.

E. It is inappropriate here because a weekly audit is latency-sensitive.

CORRECT ANSWER: A AND C
The Batch API provides roughly 50% savings and uses custom_id to correlate requests and responses (and to resubmit only failures). B is false — there is no guaranteed latency SLA and processing can take up to 24 hours; D is false — batch requests don't support multi-turn tool calling; and E is false — a weekly audit is exactly the latency-tolerant, non-blocking workload batch suits.